허건 HEO GEON

Backend Developer · 백엔드 개발자



백엔드를 만들고, 데이터로 결과를 검증합니다.

Node.js·Spring 백엔드를 중심으로 API 설계, ERD·DB 모델링, 실시간 서버, 데이터 파이프라인을 만듭니다.

엠트리센에서 백엔드 인턴으로 API·DB 구조 설계와 데이터 수집·가공 자동화를 맡았습니다. 팀·개인 프로젝트에서는 실제로 동작하는 서버 구조를 만드는 데 집중했습니다.

구현 결과를 부하 테스트와 데이터 대조로 검증해 믿을 수 있게 만드는 개발을 지향합니다.

학력 · 경력

명지대학교 컴퓨터공학과 2020 ~ 2026

엠트리센 백엔드 개발 인턴 2025 ~ 2026

수상

★ **명지대 캡스톤디자인 금상** · 음성인식 키오스크

★ **코드잇 데모데이 대상** · 조각집 (ZOGAKZIP)

기술 스택

Backend	Node.js Express Spring FastAPI
Database	MySQL MongoDB Redis
Language	Java JavaScript Python
Tools	Docker Git Swagger

연락처

Email	rjs2337@naver.com
GitHub	github.com/HUHGEON
Blog	huhgeon.github.io
Portfolio	huhgeon.github.io/portfolio

할 수 있는 것

요청이 동시에 몰리는 발급 처리에서 조건부 원자 UPDATE·Redis Lua 선점으로 재고 정합성을 지키고 개선 효과를 부하 측정 수치로 검증할 수 있습니다. — 선착순 쿠폰 발급

Spring Batch로 대용량 이력을 재생·대조하는 검증 배치를 만들고 오류를 주입한 데이터로 검증기 자체도 검증할 수 있습니다. — 선착순 쿠폰 발급

Spring WebFlux와 Redis Pub/Sub으로 여러 인스턴스에 나뉘어 연결된 사용자끼리 메시지를 중계하는 실시간 서버를 구현할 수 있습니다. — 라이브 채팅 서버

LLM 출력을 규칙 라우팅 · RAG · 스키마 강제로 제어하고 없는 장소 추천·깨진 출력·LLM 장애 시 규칙 폴백으로 대응하는 의도 해석 · 추천 서버를 설계할 수 있습니다. — 날씨 추천 앱 · 음성인식 키오스크

느린 외부 호출이 커넥션 풀을 고갈시키지 않도록 트랜잭션 경계를 나누고 장애 시 fail-open · 폴백 정책을 설계할 수 있습니다. — 날씨 추천 앱 · 라이브 채팅 서버

MongoDB text · TTL · unique 복합 인덱스와 aggregation으로 검색 · 추천 · 중복 방지 요구사항을 데이터 모델 수준에서 해결할 수 있습니다. — 블로그 플랫폼 · 음성인식 키오스크

프로젝트

01 **스마트폰 사전예약 (NOVA)** 대규모 트래픽
백엔드 저장소 커밋 **677/1,420** 으로 최다 기여, 동시성 결함 **4종**을 재현 시험으로 운영 전에 잡아 수정

02 **선착순 쿠폰 발급 시스템** 대규모 트래픽
이력 **534만** 행을 재생하는 검증 배치 설계·구현, 오류 **700건**을 주입한 데이터에서 기대 검출 **800행**을 누락 0 · 오탐 0으로 확인

03 **라이브 커머스 채팅 서버** 대규모 트래픽
서로 다른 서버에 연결된 시청자 간 채팅을 Redis Pub/Sub 전달 파이프라인으로 중계

04 **음성인식 키오스크** LLM 활용
LLM으로 intents를 추출하고 API 서버를 호출하는 오케스트레이터 구조로 NLP 서버 설계

05 **해야지 (haeyaji)** LLM 활용
로컬 LLM(EXAONE)의 오분류·없는 장소 추천을 규칙 라우팅·RAG·스키마 강제로 줄이고 시나리오 채점 **59/80**으로 남은 한계까지 확인

06 **엠트리센 인턴 · 사내 실무** 실무
원본 데이터 전처리 → 외부 서버 2곳 병렬 연동 → 저장·캐싱 백엔드 파이프라인과 Streamlit 데이터 분석·시각화 대시보드 **2종** 개발

스마트폰 사전예약 (NOVA)

백엔드 · 인증 · 상품 · 장바구니 · 드로우 · 공통 모듈 (7인 팀) · 2026.09 ~ 10 · 팀 프로젝트

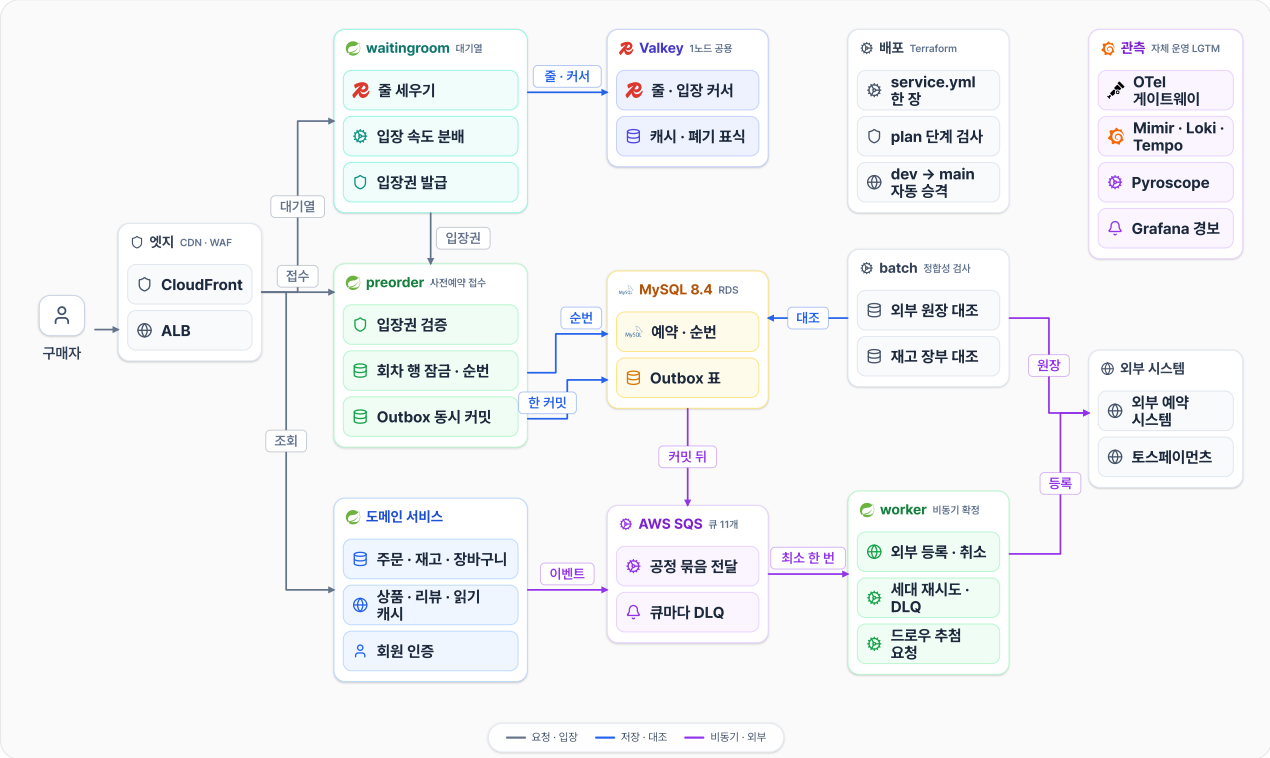
접수 완료와 외부 등록 완료를 분리해, 오픈 10초에 5,000명이 몰려도 접수는 즉시 끝내고 외부 등록은 느리거나 실패해도 끝까지 책임지는 사전예약 시스템입니다.

`Java 25` `Spring Boot 4.1` `MySQL 8.4` `Valkey` `AWS SQS` `ECS Fargate` `Terraform` `k6`

github.com/Nova-Grandis 상세 · 시연 영상 ↗

몰릴 때 접수 응답	10초에 5,000명, 서버 오류	동시 접수 데드락	백엔드 저장소 커밋
113ms	**0건**	**4/4 ⟶ 0/4**	**677 / 1,420**
7건 중앙값(64~319ms) · 5,000명이 들어오는 중 · 클라이언트 왕복 측정	10분 동안 요청 12만 8천 건 · 0.5초를 넘긴 요청 0.0102%	REPEATABLE READ 4회 전부 교착 · READ COMMITTED 4회 전부 통과	최다 기여 · 시험 4,088개 · 마이그레이션 80개

LG유플러스 유레카 최종 융합 프로젝트에서 7인 팀이 만든 스마트폰 사전예약 서비스입니다. 예약을 최종 확정하는 외부 예약 시스템은 평균 0.5초가 걸리고 20건에 1건은 실패합니다. 두 일을 한 트랜잭션에 묶으면 둘 다 무너지므로, 사용자에게 응답을 돌려주는 순간과 외부에 등록이 서는 순간을 떼어 놓고 그 사이를 Transactional Outbox · SQS · 세대 재시도가 책임지게 했습니다. 서비스 8개와 배치 2개를 ECS Fargate 에 올리고 Terraform · Atlantis 로 운영했습니다. 저는 백엔드 저장소 커밋 677/1,420 으로 최다 기여했고 회원 인증 · 상품 · 리뷰 · 장바구니 · 럭키드로우와 공통 모듈(JWT 검증 · 읽기 캐시)을 맡았습니다.



동작 흐름

- 오픈 순간 몰린 요청은 대기열이 Redis ZSET 에 줄로 세우고, 리더 한 대가 **1초**마다 상품별 입장 속도를 나눠 차례가 된 사람에게 **120초**짜리 HMAC 입장권을 준다
- 접수 서버는 입장권을 저장소 조회 없이 서명만으로 확인하고, 회차 행을 잠가 순번을 매긴 뒤 예약 · 등록 작업 · Outbox 이벤트를 한 트랜잭션에 커밋한다. 여기까지가 사용자가 기다리는 구간이다
- 커밋 직후 Outbox 가 SQS 로 발행하고, 놓친 행은 릴레이가 주워 보낸다. 큐는 **11개**이고 전부 DLQ 를 달고 있다
- 워커가 외부 예약 시스템에 등록한다. DB 를 쓰지 않고 시도 횟수를 메시지에 싣고 다니며, 응답을 모르면 다음 차례에 같은 키로 먼저 조회해 두 번 등록되지 않게 한다. 세대를 다 쓰면 DLQ 로 가고 관리자가 재처리한다

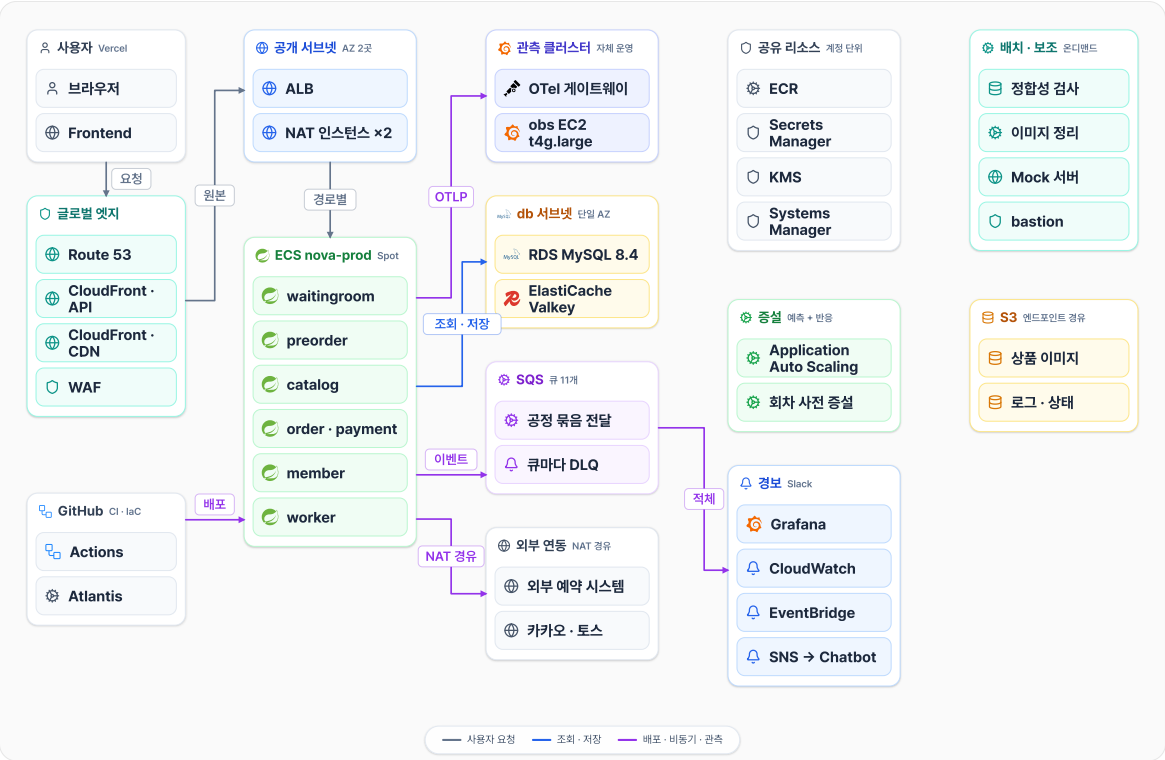
주요 기능

입장 대기열 Redis ZSET 과 Lua 로 순서를 원자적으로 매기고, 리더 한 대가 **1초**마다 상품별 입장 속도를 나눈다. 입장권은 HMAC 서명이라 접수 서버가 저장소를 보지 않고 검증한다

사전예약 접수 회원당 진행 중 예약 **1건**은 생성 칼럼 UNIQUE, 모델별 빈틈없는 순번은 회차 행 잠금, 재전송은 멱등키 UNIQUE 가 막는다. 앱이 틀려도 두 번째 행은 DB 가 거절한다

비동기 확정 처리 예약 · 작업 · 이벤트를 한 트랜잭션에 커밋하고 커밋 뒤 발행한다. 워커는 DB 를 쓰지 않고 시도 횟수를 메시지에 싣고 다니며, 응답을 모른 차례 다음에는 다시 등록하지 않고 같은 키로 먼저 조회한다

정합성 검사 한 스냅샷 트랜잭션에서 예약을 읽고 외부 원장 목록과 맞대어 **19종**을 대조한다. 외부 쪽 최근 변경은 판정을 미뤄 진행 중인 정상 건을 어긋남으로 잡지 않는다



주요 기능 (이어서)

주문 · 결제 재고는 조건부 UPDATE 와 DB CHECK 로 초과 판매를 막고, 결제는 외부 결제사를 부르기 전에 UNIQUE 로 두 번째 승인 시작 자체를 차단한다

관측 앱은 관측 저장소로 바로 보내지 않고 별도 게이트웨이를 거친다. 게이트웨이가 개인정보를 가리고, 샘플링하기 전에 요청 수와 오류율을 먼저 집계해 버려진 트레이스까지 세고, 그다음 꼬리 샘플링을 한다. 관측 서버를 다시 배포하는 동안에는 메모리 큐에 들고 최대 **15**분까지 재시도해 배포 한 번으로 데이터가 사라지지 않는다

시험 환경 모듈 **13**곳에 시험 **4,088개**와 마이그레이션 **80개**가 CI 필수 검사로 돈다. Testcontainers 로 실제 MySQL **8.4** 에 붙어 격리 수준과 갭 락까지 그대로 겪고, ArchUnit 이 모듈 경계와 잠금 순서를 코드로 강제하며, 돌연변이 시험이 시험 자체가 결함을 잡아내는지 확인한다

인프라와 배포 서비스 하나의 요구를 service.yml 한 장에 적으면 Terraform 이 보안 그룹 · IAM 역할 · DB 계정 · ALB 경로를 함께 만든다. 적지 않은 호출 대상과 큐는 보안 그룹과 IAM 에서 막히고, 최대 대수 × 커넥션 풀이 DB 연결 한도를 넘으면 plan 단계에서 적용이 멈춘다. dev 에 머지되면 검사를 다시 돌고 통과할 때만 main 으로 승격해 마이그레이션을 먼저 실행한 뒤 바뀐 서비스만 배포한다

동시성 검증 경합을 결정적으로 재현한다. 한쪽을 잠금 위에서 멈춰 세우고 PROCESSLIST 로 상대가 그 잠금을 기다리는 것을 확인한 뒤 놓아 주는 시험을 먼저 쓰고, 그다음 고친다. 재고 교착 · 빈 장바구니 **51**줄 · 리프레시 토큰 8중 회전이 모두 이 방식으로 운영 전에 잡혔다

럭키드로우 추첨 전에 외부가 받은 응모 수와 결제 완료 수를 대조해 같을 때만 추첨을 요청하고, 결과 반영은 조건부 UPDATE 하나라 한 번만 반영된다



시연 영상 · 운영 환경 시연 · 6분 03초

0:22 시연 조건 → 0:30 오픈 몰림 → 2:00 처리 서버 장애 → 2:56 두 번 등록됐을까 → 3:52 운영 현황 → 4:08 DLQ 와 재처리

huhgeon.github.io/portfolio/projects/nova/

사례 1

장바구니 주문과 결제 반영이 재고 행을 반대 순서로 잠그던 교착을 재현 시험으로 찾아 수정, 처음 의심한 원인은 범인이 아니었음을 시험으로 가려냄

문제 원인
- 앞 주문의 재고 반환과 새 주문의 재고 확보를 각각 따로 정렬해 두 번 잠가, 두 경로가 같은 옵션 두 개를 반대 순서로 집을 수 있었음
- 잠금 정렬에 쓰던 UUID.compareTo 가 부호 있는 비교라 DB 의 BINARY(**16**) 바이트 순서와 어긋날 수 있었음

해결 과정
- 반환과 확보를 옵션별 순증감 하나로 합쳐 바이트 오름차순으로 한 번만 잠그게 변경
- 비교기를 부호 없는 비교로 바꿔 DB 의 ORDER BY option_id 와 같은 순서를 보장

결과
- 교착을 실제로 만든 원인은 '따로 두 번 정렬' 쪽이었고, compareTo 는 상위 비트가 1인 id 에서만 드러나는 아직 터지지 않은 결함이었음
- 개인 기술 정리 문서에 compareTo 를 재현된 원인으로 적어 둔 것을 코드 기준으로 정정

사례 2

빈 장바구니에 동시에 담으면 상한 50줄을 넘기던 결함을 멈춤 주입 시험으로 재현해 수정

문제 원인
- 같은 회원의 장바구니 쓰기는 그 회원의 줄 전부를 FOR UPDATE 로 잠가 직렬화하는데, 빈 장바구니에는 잠글 줄이 없어 아무것도 잠기지 않음
- 빈 장바구니를 읽고 멈춘 담기와 그사이 **50**줄을 담은 요청이 겹치면 **51**줄이 됨

해결 과정
- 넣은 뒤 같은 트랜잭션에서 다시 잠가 세고 **50**을 넘으면 롤백
- 같은 옵션·보증 조합은 UNIQUE, 수량은 CHECK 1~99 로 DB 가 최종 보장

결과
- 동시 담기에서 **400** 응답과 **50**줄로 끝남

사례 3

읽기 캐시 세대 키가 사라지면 지운 줄 알았던 옛 값이 되살아나던 것을 Valkey 재현으로 찾아 수정

문제 원인
- 목록·검색은 조건마다 키가 달라 쓰기 한 번에 키를 전부 찾아 지울 수 없어, 값 키에 세대 번호를 넣고 커밋 뒤에만 번호를 올리는 방식을 씀
- 세대 키가 제거 정책이나 노드 재시작으로 사라지면 0 부터 다시 시작해, 이미 저장돼 있던 옛 세대의 값이 다시 유효해짐
- 스냅샷에서 DUMP·RESTORE 로 되돌린 경우에도 같은 문제가 발생

해결 과정
- 없는 세대는 0 이 아니라 현재 시각에서 시작하게 하고, 본 번호보다 작아진 세대는 본 번호 + 1 로 올림
- 세대 올리기가 실패하면 메모리에 미뤄 두고 재시도

결과
- 세대 소실과 스냅샷 복원 뒤에도 옛 값이 나오지 않음
- 같은 계열 작업에서 목록·카테고리 캐시를 Redis 에서 인스턴스 메모리로 옮김 (같은 조건 측정 목록 **1.05ms → 0.005ms**, 카테고리 **0.40ms → 0.001ms**)

사례 4

부하 시험에서 catalog 네 대 중 한 대만 막히는 것을 태스크별 지표로 확인해 ALB 분배를 바꾸고, 대시보드 CPU 50%가 실제로는 포화였다는 것을 팀이 함께 찾아냄

문제 원인
- 증설 뒤에도 느린 응답이 남아 처음에는 증설이 늦은 것으로 판단
- 태스크별로 보니 네 대 중 한 대만 톰캣 처리 중 요청 **200**(스레드 상한)에 걸려 막혔고 나머지 세 대는 **0.5초** 초과 **0%**
- 팀이 같이 확인한 결과 Fargate 가 **1 vCPU** 태스크에 CPU **2개**를 보여 줘서 JVM CPU 지표가 포화에서도 **0.50** 에 멈춰 있었음

해결 과정
- ALB 분배를 라운드로빈에서 처리 중 요청이 적은 태스크로 보내는 방식으로 변경
- 팀이 이어서 catalog 를 **2 vCPU** 로 올리고 -XX:ActiveProcessorCount 를 고정, 공개 GET 을 CloudFront 가 캐시하게 함

결과
- 변경 뒤 실행에서 분 단위 p95 9~29ms. 같은 PR 에 CDN 캐시도 함께 들어가 ALB 분배 단독 효과는 따로 떼어 적지 않았다
- 합산 지표만 봤으면 네 대가 고르게 일하는 것으로 보였다. 태스크 단위로 쪼개 본 것이 원인을 찾은 지점이다

버전	선택 이유	트레이드오프	수치
v1 **오토스케일링에 맡김** 구현	오픈 순간의 유입을 지표 기반 자동 증설이 받아주기를 기대 CPU·요청 수 지표로 ECS 서비스를 자동 증설. 운영 비용이 들지 않고 평상시에는 충분	증설 판단이 1분 지표 기준이라 10초짜리 몰림에는 반응 자체가 늦음 → 몰림이 끝난 뒤 증설되는 구조로는 오픈 순간을 받을 수 없음	**60초** 몰림이 끝난 뒤에야 새 태스크가 뜸 (**10**-22 시험)
v2 **회차 일정에 맞춘 사전 증설** 구현	지표를 보고 늘리는 방식은 10초 몰림보다 항상 늦음 회차 오픈 15분 전부터 예상 방문자 수를 기준으로 대기열·접수 서비스 대수를 미리 올림. 몰림이 오기 전에 용량이 서 있게 하는 것이 유일한 대책이라고 판단	예상 방문자 수가 비어 있는 회차는 1대로 계산돼 아무것도 늘지 않음. 실제로 그 상태에서 대기열 1대가 포화해 503 이 210건 발생 → 앞단 대수만 늘려서는 뒤쪽 접수 서버가 받을 수 있는 양을 모름	대기열 **2 vCPU** 1대가 초당 약 **500건** 진입에 CPU **87**~98%
v3 **접수 가용량 기반 입장 속도** 구현	대기열이 뒤쪽 접수 서버 용량을 모른 채 사람을 들여보냄 접수 서버 태스크가 공유 저장소에 초당 처리 가능 수를 적고, 대기열 리더가 그 합으로 전체 입장 속도 상한을 1초마다 다시 계산	입장 속도가 접수 서버의 자기 보고에 기대므로, 보고값이 실제 처리량과 어긋나면 입장 속도도 같이 어긋난다	입장 속도 **20** 으로 **10초**에 **5,000명**을 넣어 서버 오류 **0건** · 분 단위 p95 **29**~61ms (**1회**)

오픈 몰림 부하 측정 ms

5,000명을 10초에 밀어 넣은 시험(k6, 도착률 고정 초당 500명)의 1분 단위 진행 값입니다. 대기열 입장 속도를 운영 설정과 같은 초당 20명으로 두고 처리 기한을 7분 16초로 계산했습니다.

경과	초당 처리	p95	p99	5xx	남은 VU
1:00	746.3	39ms	177ms	0.00%	4,082
2:00	505.8	29ms	181ms	0.00%	2,887
3:00	452.4	30ms	181ms	0.00%	1,687
4:01	**316.5**	**61ms**	**211ms**	**0.00%**	**286**

부하 발생기 k6 태스크 6대(4 vCPU) · 대기열 8대 · 접수 5대(촬영 전 수동 상향) · RDS db.t4g.large 단일 AZ · Valkey cache.t4g.small 1노드. 같은 10분 구간을 Grafana 로 보면 요청 12만 8천 건, 5xx 0건, 0.5초 초과 0.0102% 입니다. 이 회차의 외부 Mock 은 정상 조건(지연 200ms · 실패 0)이었고, 외부 지연과 실패는 따로 걸어 재처리를 확인했습니다.

배운 점

가장 많이 배운 것은 계기를 먼저 의심해야 한다는 것입니다. 대기열이 느릴 때 공용 저장소를 두 번 의심했지만 두 번 다 측정으로 배제됐고 원인은 앱의 CPU 포화였습니다. 대시보드가 CPU 50% 라고 하던 태스크는 실제로는 포화 상태였고, '1만 명 중 0명 처리'라는 결과는 시스템이 아니라 수집 설정이 라벨을 잘라낸 탓이었습니다. 숫자가 이상하면 시스템보다 재는 쪽을 먼저 보는 습관이 생겼고, 그래서 이 페이지의 수치에도 측정 조건을 같이 적었습니다.
반대로 동시성 결함은 운영에서 터지기 전에 재현 시험으로 먼저 잡을 수 있었는데, 멈춤을 주입해 경합을 결정적으로 재현하는 시험을 먼저 쓰고 고치는 순서가 효과적이었습니다.

선착순 쿠폰 발급 시스템

조장 · 배치 · 검증 · 스키마 (5인 팀) · 2026.08 · 팀 프로젝트

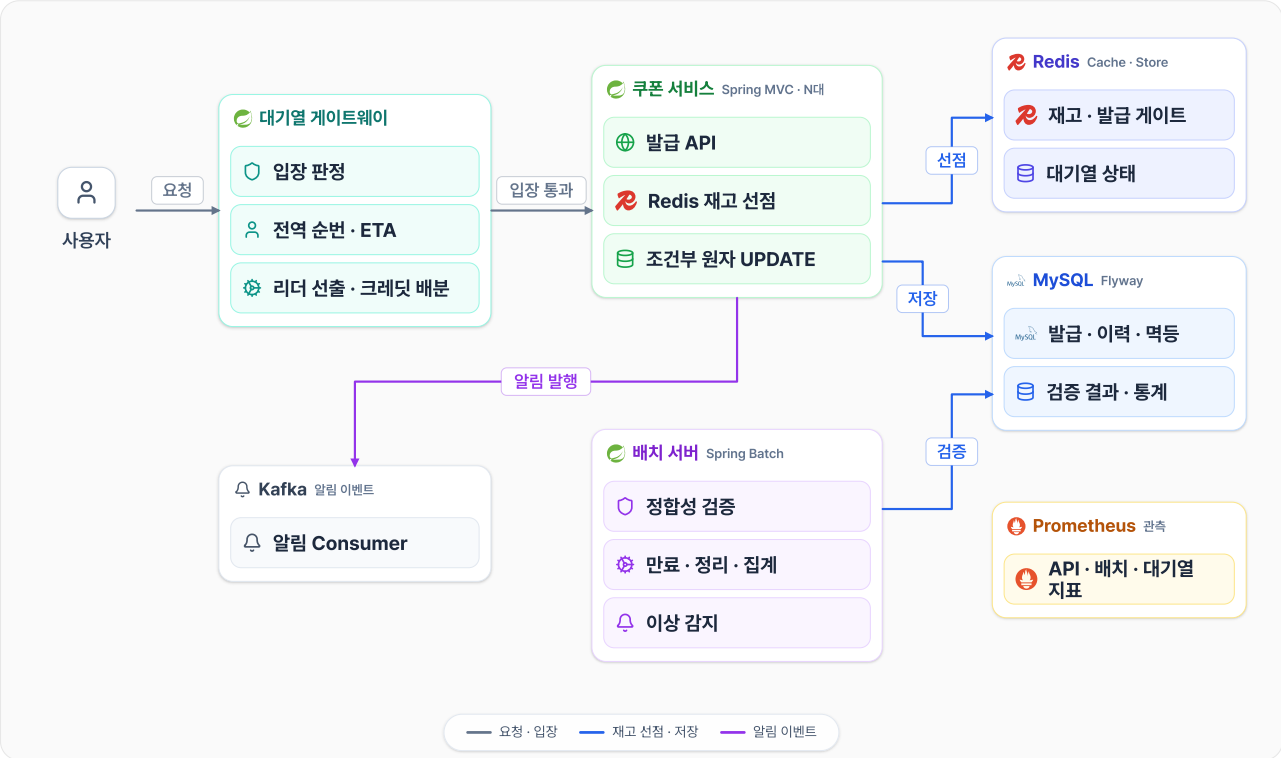
선착순 요청이 한꺼번에 몰려도 한정 수량 쿠폰을 정확히 발급하고, 그 결과를 데이터로 검증하는 시스템입니다.

`Java 21` `Spring Boot` `Spring Batch` `MySQL` `Redis` `Kafka` `k6`

github.com/coupon-yaho 상세 · 시연 영상 ↗

오염 데이터 기대 검출	500 req/s 중앙값 1초 이상 회차	500 req/s 성공 응답 p99	초과 발급
800 / 800	**4/7 → 0/7**	**2,240 → 802ms**	**0건**
700건 주입(1개 유형이 규칙 2개에 걸려 기대 800행) · 누락 0 · 오탐 0	7회 중 성공 응답 중앙값이 1초 이상인 횟수	성공 응답 기준 · 중앙값 1초 이상 회차 제외 · p95 2,160 → 641ms	105회차 전체

LG유플러스 유레카 백엔드 종합 프로젝트에서 5인 팀이 만든 통신사 브랜드데이 선착순 쿠폰 발급 시스템(쿠폰 야호~)입니다. 팀은 병목을 측정하면서 MySQL 락, 조건부 원자 UPDATE, Redis Lua 선점, 적응형 대기열 순으로 구조를 바꿔 왔고, 저는 조장으로서 검증 배치, 시드 생성기, 부하 시험 결과의 DB 대조를 맡았습니다.



동작 흐름

- 사용자 요청이 NGINX를 거쳐 대기열 게이트웨이(WebFlux)로 들어오면 즉시 종결 · 무대기 통과 · 대기열 진입 중 하나로 판정
- 통과한 발급 요청은 쿠폰 서비스가 재고를 판정하고 발급과 멱등 기록(DONE)을 한 트랜잭션에서 확정. 재고 판정은 v1.2가 조건부 원자 UPDATE, v2.1이 Redis Lua 선점
- MySQL이 발급 · 상태 이력 · 사용 실적 · 멱등 응답을 저장하고 UNIQUE(coupon_id, member_id)로 1인 1매를 최종 보장
- 발급 결과 알림은 Kafka로 발행

주요 기능

선착순 쿠폰 발급 멱등키(UUID)로 재시도해도 한 장만 발급, 조건부 원자 UPDATE와 UNIQUE(coupon_id, member_id)로 초과·중복 발급 차단

적응형 대기열 평시엔 바로 통과, 혼잡하면 백엔드 가용량에 맞춰 입장량을 조절하고 순번·예상 대기시간·입장 토큰 제공

정합성 검증 배치 검증 규칙 **6종**이 발급 이력을 런타임과 같은 상태 머신으로 재생해 재고 카운터·사용 실적과 대조

대용량 시드 생성기 회원 **100만**·발급 **300만**·이력 **534만** 행을 정상셋과 오류를 심은 오염셋 두 벌로 생성, 시드 고정 RNG로 같은 시드면 같은 데이터 재현



시연 영상 · 대기열 입장부터 발급 관제 · 검증 배치까지

대기열 입장 → 발급 관제 → 검증 배치

huhgeon.github.io/portfolio/projects/coupon-yaho/

사례 1

오류 700건을 심은 데이터셋으로 검증 배치가 실제로 오류를 찾아내는지 확인, 누락 0 · 오탐 0

문제 원인
- 검증 배치 결과 **0건**이 오류가 없어서인지 검증기가 못 찾아서인지 구분할 근거가 없음
- 검출 개수만 비교하면 누락과 오탐이 같은 수만큼 섞여도 합계가 기대치와 같아져 통과
- 회원 **100만** · 발급 **300만** 규모를 작은 MySQL 컨테이너에 적재해야 했고 이메일을 난수로 만들면 해시 인덱스 충돌로 적재 전체가 실패할 수 있음

해결 과정
- 시드 생성기로 정상셋(CLEAN)과 오류 7유형 × **100건** = **700건**을 심은 오염셋(CORRUPT) 두 벌을 만들고 기대 검출 결과도 기록
- 기대 검출이 **800행**인 이유: 7유형 중 '사용 취소 이중 기록' **100건**은 재고 불일치(V1)와 불법 상태 전이(V4) 두 규칙에 동시에 걸려 **200행**으로 잡힘 → **600** + **200** = **800행**
- 검출 결과를 (finding_type, target_key) 집합 단위로 양방향 대조해 누락과 오탐을 따로 집계
- 시드 고정 RNG와 Feistel 순열로 겹치지 않는 값을 만들어 UNIQUE 충돌 0 보장, 보조 인덱스 없이 적재한 뒤 제약을 한 번에 생성

결과
- CLEAN: 회원 **100만** · 발급 **300만** · 이력 **534만** 행에서 검증 규칙 **6종** 모두 **0건**, 이력 전체 리플레이 **57초**
- CORRUPT: 기대 **800행** → 검출 **800** · 누락 0 · 오탐 0
- CLEAN 생성 · 적재 **199.7초**, 제약 생성 **81초**

사례 2

팀에서 재고 잠금을 FOR UPDATE에서 조건부 원자 UPDATE로 바꿔 500 req/s 중앙값 1초 이상 회차 4/7 → 0/7, 성공 응답 p99 2,240ms → 802ms

문제 원인
- 재고 행이 회차당 하나뿐이라 모든 발급이 같은 잠금을 순서대로 통과하고 FOR UPDATE는 조회 시점부터 INSERT까지 임계구간에 넣어 잠금 구간이 김
- 도착률을 올리자 **450 req/s**부터 성공 응답 중앙값이 **1초** 이상인 회차가 나왔고(**1/7**), **500 req/s**에선 **7회** 중 **4회**로 늘어남
- 지연이 커진 회차도 5xx 없이 느려지기만 해서 가용성 지표만으로는 문제를 잡지 못함

해결 과정
- 검사와 차감을 UPDATE … WHERE active_count < total_quantity 한 문장으로 합쳐 잠금이 UPDATE에서 시작되게 하고 영향 행이 0이면 소진으로 판정
- 1인 1매는 UNIQUE(coupon_id, member_id)로 DB에서 최종 보장
- 이후 재고 판정을 Redis Lua 선점으로 옮기고 앞단에 적응형 대기열을 두는 구조로 확장 (아래 버전별 발전 과정)

결과
- **500 req/s** 성공 응답 p99 **2,240** → **802ms**, p95 **2,160** → **641ms**(중앙값 **1초** 이상 회차는 지연 집계에서 제외), 중앙값 **1초** 이상 회차 **4/7** → **0/7**
- **105회차** 전체에서 초과 발급 **0건**

사례 3

측정 환경에 다른 컨테이너가 함께 돌던 것을 발견해 이전 수치를 모두 폐기하고, 외부 요인을 통제한 환경에서 세 잠금 구현을 105회차 재측정

문제 원인
- 첫 측정 때 다른 프로젝트 컨테이너 **6개**가 함께 돌고 있어 같은 FOR UPDATE 구현의 **300 req/s** p99가 **6,873ms**로 부풀려짐

해결 과정
- 외부 컨테이너를 모두 내리고 회차마다 외부 컨테이너 수와 호스트 CPU 기록
- 세 이미지를 번갈아 실행해 시간대 편향을 없애고 회차마다 예열 후 측정

결과
- 같은 조건 FOR UPDATE **300 req/s** p99 **6,873** → **222.72ms**로 외부 영향 제거, 이전 수치는 전부 폐기
- **500 req/s** 중앙값 **1초** 이상 회차: FOR UPDATE **4/7** · 조건부 UPDATE **0/7** · Redis 분리 실험 **0/7**

버전	선택 이유	트레이드오프	수치
v1.1 **비관적 락** 구현	동시 요청이 회차당 재고 행 하나를 두고 경쟁해 정합성 확보가 먼저 SELECT … FOR UPDATE로 재고 행을 잠근 뒤 검사·차감. DB가 정확성을 보장하는 가장 단순한 방식이라 이후 버전 비교의 기준선으로 삼음	모든 발급이 같은 잠금을 순서대로 통과하고 INSERT까지 임계구간에 들어가 커넥션 풀을 늘려도 대기가 줄지 않음 → 잠금 구간이 길어 포화 지점이 400~450 req/s에 머묾	**450 req/s** 중앙값 **1초** 이상 회차 **1/7** · **500 req/s** 성공 p95 **2,160ms** · p99 **2,240ms** · 중앙값 **1초** 이상 회차 **4/7**
v1.2 **조건부 원자 UPDATE** 구현 · 제품 코드	요청마다 잠금 조회와 차감 UPDATE로 두 번 왕복하며 잠금을 오래 잡음 UPDATE … WHERE active_count < total_quantity 한 문장으로 검사와 차감을 합치고 영향 행이 0이면 소진 판정. 잠금이 UPDATE에서 시작돼 구간이 짧아짐	재고 행 하나를 두고 경쟁하는 구조와 요청별 동기 DB 트랜잭션은 그대로 남음 → 경합 지점이 여전히 DB의 재고 행 하나라 재고 카운트를 DB 밖으로 옮김	**500 req/s** 성공 p95 **641ms** · p99 **802ms** · 중앙값 **1초** 이상 회차 **0/7**
v2.1 **Redis Lua 원자 선점** 구현	DB에 직렬 구간이 남아 있으면 커넥션을 늘려도 처리량이 늘지 않음 Lua 스크립트 5종(claim · complete · compensate · restore · reclaim)이 회차 상태 · 등급 · 1인 1매 · 재고를 한 번에 판정하고 차감. 발급은 선점 → 트랜잭션 → 완료 CAS 세 단계로 분리	저장소가 둘로 나뉘어 선점 직후 중단되면 Redis가 DB보다 앞서감. 요청 토큰으로 보상, Redis 유실 시 DB 기준으로 재구성, Redis↔DB 격차 모니터링 지표 추가 → DB 경합은 줄었지만 순간 유입 자체와 건별 영속화는 그대로 남음	재고 UPDATE를 별도 트랜잭션으로 분리한 실험(v2-split): **500 req/s** 성공 p95 **562ms** · p99 **748ms** · 중앙값 **1초** 이상 회차 **0/7** · 발급당 DB 쓰기 **7행** → **3행** (실험 · 미커밋)
v2.2 **적응형 대기열** 핵심 경로 구현	순간적으로 몰리는 유입이 애플리케이션과 DB에 함께 과부하를 줌 게이트웨이를 별도 서비스로 분리한 이유는 세 가지. 경계(입장만 담당, 재고 차감은 쿠폰 서비스만) · 스택(입장은 WebFlux, 발급은 MVC + JPA라 한 프로세스에 두면 블로킹이 이벤트 루프를 막음) · 장애 격리(뒷단이 멈춰도 대기열 유지)	입장은 순서만 보장할 뿐 발급은 보장하지 않음. 컴포넌트가 늘고 성공 요청마다 동기 DB 커밋도 여전히 필요	페이즈 게이트 7단계 통과 · 요청 경로 Redis 명령 **0건** · 오버헤드 p99 < **5ms** · 크레딧 초과 배분 **0/10만** 회 · 매진 후 동시 조회 **1만** → 뒷단 **1건**

잠금 방식별 부하 측정 ms

같은 커밋에서 재고 잠금 방식만 바꿔 재고 10,000 · 요청 20,000 조건으로 도착률마다 7회씩 측정했습니다. p95 · p99(ms)는 성공 응답 중앙값이 1초 미만인 회차들의 중앙값이며, 1초 이상인 회차 수는 오른쪽 열에 따로 표시했습니다.

도착률	구현	p95	p99	1초 이상 회차
300/s	v1.1 FOR UPDATE	50.10	222.72	0/7
	v1.2 조건부	**28.12**	**158.64**	**0/7**
400/s	v1.1 FOR UPDATE	409.35	481.83	0/7
	v1.2 조건부	**190.41**	**264.03**	**0/7**
450/s	v1.1 FOR UPDATE	562.01	648.15	1/7
	v1.2 조건부	**368.66**	**575.49**	**0/7**
500/s	v1.1 FOR UPDATE	2,159.79	2,239.89	4/7
	v1.2 조건부	**640.92**	**802.16**	**0/7**

k6 constant-arrival-rate · 로컬 Docker(API 2대 · MySQL 단일) · 외부 컨테이너 0개 · 회차마다 상태 롤백. 1초 이상 회차는 성공 응답 중앙값이 1,000ms 이상인 회차로, 지연 집계에서 제외했습니다. 따라서 FOR UPDATE 450 · 500 req/s의 p95 · p99는 나머지 회차만의 값이며 실제 꼬리 지연은 더 나쁩니다.

배운 점

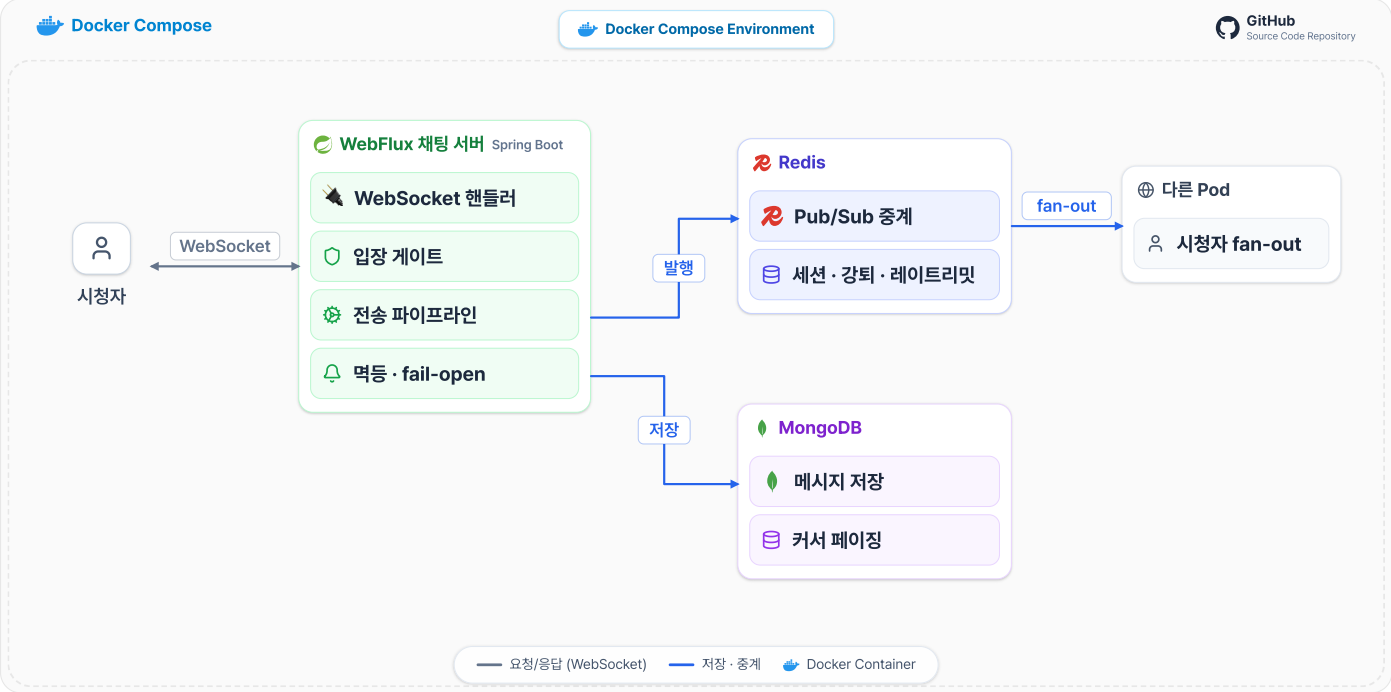
평상 부하(300 req/s)에서는 세 구현의 성공 응답 중앙값이 3.4~4.0ms로 차이가 드러나지 않았습니다. 개선 효과는 평소 지연을 줄인 것이 아니라 포화 지점을 끌어올린 데 있었고, 이 차이는 지표를 제대로 고르고 측정 환경을 통제해야만 보인다는 것을 배웠습니다. 검증도 마찬가지로 '0건'이라는 결과는 오류를 심어 두고 전부 찾아냈을 때만 믿을 수 있다는 기준을 세웠습니다.

라이브 커머스 채팅 서버

sapari(라이브 스트리밍·커머스) 채팅 서버 담당 · 진행 중 · 도메인 확장 예정 · 2026.06 ~ 진행 중 · 팀 프로젝트

여러 Pod에 흩어진 시청자에게 메시지를 실시간으로 전달하는 라이브 채팅 서버입니다.

`Java 21` `Spring WebFlux` `WebSocket` `Redis` `MongoDB` github.com/sago-panda/sapari-be/tree/feature/SPR-16 상세 페이지 ↗



문제

- WebSocket은 stateful이라 시청자가 여러 Pod에 나뉘고, A Pod에서 보낸 메시지를 B Pod 시청자에게도 실시간으로 전달해야 함
- 단일 이벤트 루프를 블로킹하지 않으면서 재전송·중복 프레임·파이프라이닝을 이용한 레이트리밋 회피를 막아야 함
- 채팅 본문·clientMsgId·프레임 같은 신뢰할 수 없는 입력을 이용한 욕설 우회·로그 위조를 막아야 함

결과

- 어느 Pod에 접속해도 같은 방 메시지를 받는 stateless 수평 확장 구조, 한 JVM에서 Pod **2개**를 모사한 TestContainers(Redis) 통합 테스트로 Pod 간 전달 확인 (Pub/Sub 특성상 전달은 best-effort이고 재접속 시 누락 복구는 미구현)
- Redis 장애(fail-open)·poison-message(개별 skip)·재전송(멱등 처리) 세 상황 모두에서 전송 파이프라인이 멈추지 않도록 구성

해결

- 비용 0인 검사는 앞에, Redis I/O는 뒤에 두는 순서(방 생존 → 검증 → 권한 → 강퇴 → 레이트리밋 → 욕설 마스킹 → 저장 → 발행)로 전송 파이프라인 구성
- Redis Pub/Sub 패턴(chat:pubsub:*)을 hot 스트림으로 상시 구독해, 방별 lazy 구독에서 생기는 '구독 완료 전 메시지 유실' 레이스 없이 Pod 간 fan-out
- Aho-Corasick 1-패스 욕설 탐지로 특수문자 삽입·인접 토큰 결합 우회까지 잡고, '시발점' 같은 일반 단어는 화이트리스트로 보존
- clientMsgId + Mongo partial unique index로 멱등 처리(persist-then-publish)해 재전송 시 중복 발행을 막고 기존 메시지로 ack
- Redis 장애로 강퇴·레이트리밋 조회가 실패해도 채팅 허용(fail-open), onErrorResume 람다 안 throw 금지와 시간 기반 로그 스로틀로 fail-open 정책이 뒤집히지 않게 함
- chat-api(계약) ↔ chat-core(도메인·인프라)를 헥사고날 구조로 분리하고 ArchUnit으로 강제, 인바운드는 concatMap으로 순차 처리

음성인식 키오스크

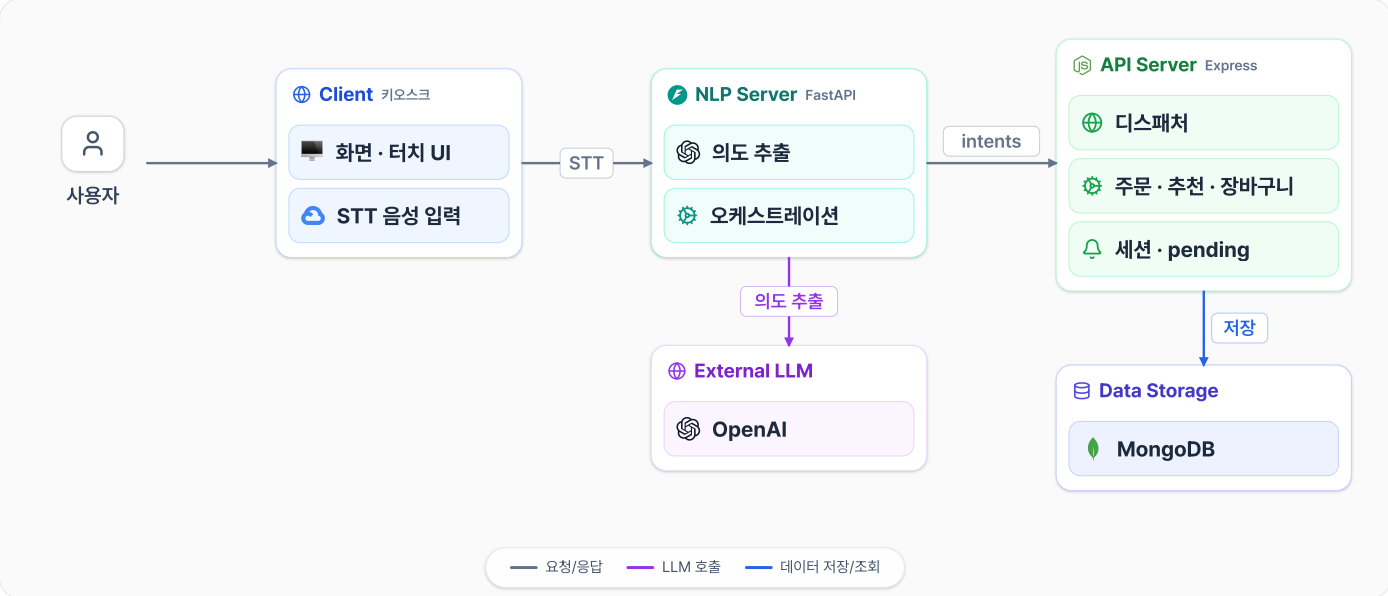
★ 명지대 캡스톤디자인 금상

NLP 서버 설계·주도 · API 연동 · 2025.03 ~ 2025.06 · 팀 프로젝트

자유발화 한 문장에 담긴 여러 요청을 순서대로 처리하는 음성 주문 백엔드입니다.

`FastAPI` `gpt-4o-mini` `Express` `MongoDB` `Swagger`

github.com/Say-It-It-s-OK 상세 · 시연 영상 ↗



문제

- 한 문장에 섞인 여러 요청('카페라떼 빼고 아메리카노 추가하고 결제')을 순서대로 실행해야 함
- 주문 시 필수 옵션(온도·크기 등)이 빠져도 바로 실패로 끝내지 않고 다음 발화로 채워야 함

결과

- 복합 요청 처리: 한 문장에 담긴 여러 의도를 순서대로 실행
- 멀티턴 처리: 빠진 옵션을 후속 발화로 채우며 주문 진행
- 추천부터 장바구니 수정·결제까지 이어지는 키오스크 백엔드를 팀 프로젝트로 완성
- 결제 시 메뉴별 수량을 그룹핑해 Orders 컬렉션(MongoDB Atlas)에 실제 저장, API는 Swagger로 문서화 (팀 API 서버)

해결

- LLM 시스템 프롬프트에 intents **9종**·filters·옵션 정규화(샷 표현·온도 수식어 분리 등) 규칙을 정의하고 gpt-4o-mini로 JSON 추출
- intents가 있으면 query.sequence로 묶고 API 서버가 for-loop로 순차 실행, 중첩 응답은 results만 펼쳐(flatten) 병합
- 옵션이 빠지면 pending으로 세션에 보관, 옵션만 담긴 후속 발화가 오면 남은 옵션을 계산해 다 채워졌을 때 장바구니에 담기 (팀 API 서버와 연동)
- 추천은 MongoDB aggregation으로 주문↔메뉴 $lookup·인기순 집계, 이미 추천한 항목은 $nin으로 제외해 중복 추천 방지 (팀 API 서버)
- menus·query·recommend·order·cart API를 Swagger(swagger-jsdoc)로 문서화해 프론트·NLP와 연동 규격 공유 (팀 API 서버)



시연 영상 · 음성 대화로 추천부터 결제까지

인기 메뉴 보여 줘 → 아이스 아메리카노 미디움 사이즈로 주문해 줘 → 미디움으로 해 줘 → 결제해 줘

huhgeon.github.io/portfolio/projects/voice-kiosk/

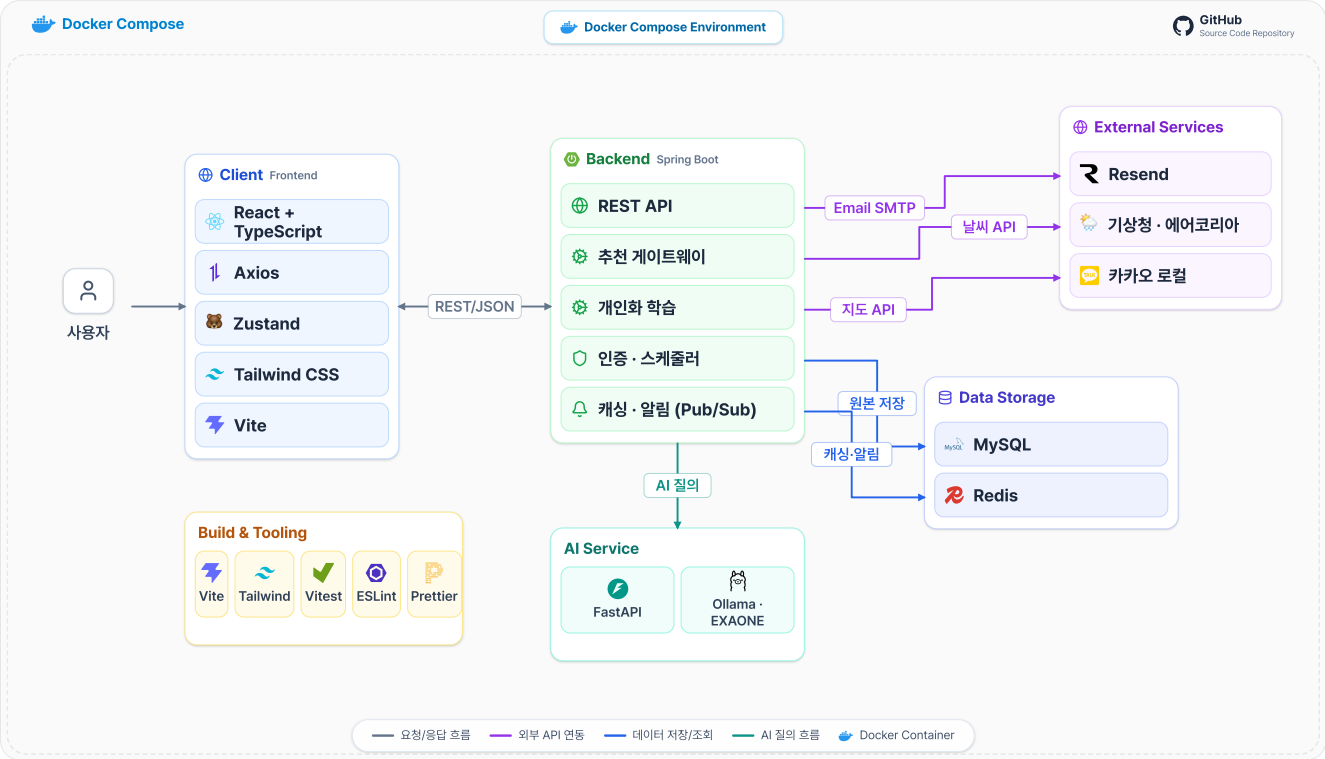
해야지 (haeyaji)

NLP 추천 서버 단독 개발 · 백엔드 날씨/추천/개인화/알림 도메인 · 2026.06 ~ 2026.07 · 팀 프로젝트

작은 로컬 LLM에 규칙과 RAG를 붙여 '오늘 뭐 할지' 추천이 덜 들쭉날쭉하게 만든 추천 서버입니다.

`FastAPI` `Ollama · EXAONE` `Spring Boot` `MySQL` `Redis` `QueryDSL`

github.com/haeyaji 상세 · 시연 영상 ↗



문제

- 작은 로컬 LLM(**2.4**~7.8b)의 막연한 발화 오분류·없는 가게를 지어내는 환각·구조화 JSON 형식 깨짐
- LLM 응답이 최대 **20**~45초 걸리는 동안 트랜잭션·커넥션을 붙잡으면 커넥션풀 고갈, 외부 API 호출 폭주와 알림 유실도 막아야 함

결과

- nlp 시나리오 자동 채점 **59/80** 통과 (**2026**-07-06 기준: 날씨·위치 **19/20**, 막연한 요청 **17/20**, 상세 추천 **17/20**, 도메인 밖 거절 **6/20**)
- 도메인 밖 거절 실패 **14건**(요리법·번역·운세 등에서 거절이 동작하지 않음)을 확인해 LLM 호출 전 규칙 라우팅을 보강했고, 7.8b 모델로는 작정하고 시도하는 사회공학을 완전히 막을 수 없다는 한계를 PR에 명시 (보강 후 동일 평가셋 재측정 결과는 기록 없음)
- LLM·네트워크 없이 도는 결정적 테스트(nlp **119개**·be **36개**)로 규칙·라우팅·폴백·날씨 변환·알림 멱등성 회귀 검증

해결

- LLM은 활동·검색어만 정하고, 카카오 후보 목록을 넣어 그 안에서만 고르게 하는 RAG로 없는 장소 추천 방지(후보에 없으면 장소 연결 해제)
- 막연한 표현·재요청·부정·인사·도메인 밖 거절·프롬프트 인젝션은 LLM 호출 전에 규칙으로 확정, 카테고리는 날씨×시간대 가점으로 점수화
- Ollama 호출에 format=<Pydantic schema>로 출력 스키마를 강제하고, 실패·타임아웃·깨진 JSON에도 **500** 대신 규칙 폴백 반환
- 추천 게이트웨이 트랜잭션을 분리해 느린 nlp 호출 전에 DB 조립과 커넥션 반납을 끝내고, LLM 응답 중 커넥션풀 고갈 방지
- 고른 것 +2 / 함께 노출됐지만 안 고른 것 -0.05를 (날씨×시간대) 맥락별로 원자적으로 누적하고 주간 decay를 적용하는 개인화 가중치 학습
- 위경도→격자·중기 지역코드·최근접 측정소 변환, 발표시각 자동 계산, Redis 캐시, 외부 API 장애 때도 응답을 유지하는 기상청·에어코리아 날씨 중계 파이프라인 구축



시연 영상 · 날씨 기반 장소 추천부터 일정 추가까지

날씨 홈 → 지도 추천 → 추천 도우미 → 일정 추가

huhgeon.github.io/portfolio/projects/haeyaji/

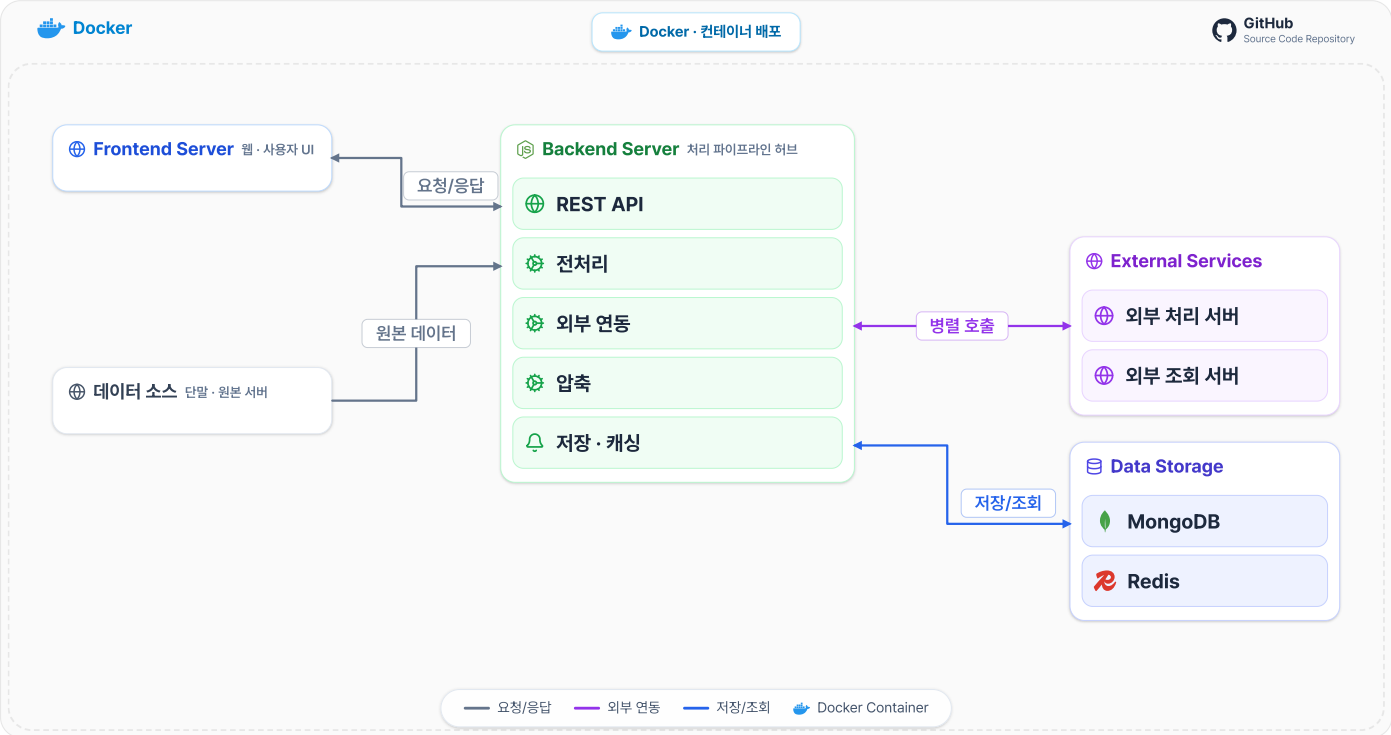
엠트리센 인턴 · 사내 실무

백엔드 · 데이터 분석 자동화 · QA (사내 프로토타입) · 2025.08 ~ 2026.01 · 엠트리센 인턴

백엔드 프로토타입 개발부터 데이터 분석 자동화, QA까지 맡은 사내 실무입니다.

`Node.js` `Express` `Python` `Streamlit` `pandas` `MongoDB` `Redis` `Jest`

사내 · 비공개 저장소 상세 페이지 ↗



사내 실무 · 기술 중심

백엔드 프로토타입 · 처리 파이프라인

원본 데이터 전처리(파일명 파싱·외부 조회 결과와 교차 검증) → 외부 서버 2곳 병렬 호출 → FFmpeg 압축 → 테넌트별 DB 저장·Redis 캐싱으로 이어지는 백엔드 파이프라인 구현. 처리·조회·캐시·외부 연동을 서비스 레이어로 분리하고, 목록 조회는 커서 페이지네이션과 쓰기 시 캐시 무효화로 처리

`Node.js` `Express` `MongoDB` `Redis` `FFmpeg` `Docker`

데이터 분석·시각화 대시보드 ①

기획팀에서 부탁받은 조사용 엑셀 작업이 반복된다는 점을 보고 스스로 자동화. 엑셀 데이터 로드·가공·통계 분석(Kruskal-Wallis·Pearson 상관)·차트 시각화를 하는 Streamlit 대시보드로 수작업 분석 대체. 데이터 로드·가공·시각화는 유틸 모듈로 분리

`Python` `Streamlit` `pandas` `numpy` `matplotlib` `scipy`

데이터 분석·시각화 대시보드 ②

같은 기획팀 조사 업무에서 기간·조건별 엑셀 집계·분석을 Altair 차트 기반 Streamlit 대시보드로 자동화

`Python` `Streamlit` `pandas` `numpy` `Altair`

테스트 · QA

- 백엔드: Jest·supertest로 테스트 케이스 **194개** 작성(DB·캐시·외부 호출은 mock으로 격리), 테스트하기 쉽도록 핸들러와 비즈니스 로직을 분리하는 리팩터링 진행
- 타사 앱 QA: 사내에서 다른 회사 앱을 직접 테스트하고 결과를 테스트 보고서로 정리·보고하는 업무를 자주 담당

※ 도메인·세부 기능·정량 성과는 대외비로, 사용 기술과 구조만 기재.